The problem

Problem Statement: The Friction of Shared Expenses

Executive Summary

Managing group expenses is a socially awkward and mathematically tedious process. Despite the frequency of group activities—such as traveling, sharing an apartment, or dining out—the methods for tracking, calculating, and settling debts remain fragmented, leading to financial loss and strained personal relationships.

Core Challenges

1. Financial Leakage and Inaccuracy

Individuals who cover the initial bill often end up losing money due to "invisible costs." These include:

- **Uncalculated Add-ons:** Tips, taxes, and service fees are frequently omitted during manual splits.
- **Rounding Errors:** Inconsistent methods for handling decimals lead to small but cumulative losses.
- **Missing Records:** The lack of immediate receipt digitization results in forgotten expenses and lost proof of payment.

2. Logistical and Mathematical Complexity

Splitting costs is rarely a simple division by the number of participants. Complexity arises from:

- **Mixed Payment Methods:** Groups often use a combination of cash, digital wallets, and credit cards, making it hard to track the flow of money.
- **Unequal Consumption:** Traditional "split-even" methods are unfair when participants have vastly different consumption patterns (e.g., someone not ordering alcohol or appetizers).
- **External Constraints:** Many establishments refuse to split checks, forcing one person to take the financial risk for the entire group.

3. Social Friction and Communication Breakdown

The most significant burden is psychological. The "debtor-creditor" dynamic creates several social pain points:

- **The "Debt Collector" Paradox:** People often feel embarrassed or "cheap" when asking friends for their money back, leading them to abandon the debt altogether.
- **Information Asymmetry:** Discrepancies in memory regarding who paid for what lead to misunderstandings and mistrust.

- **Forgetfulness and Procrastination:** Without automated reminders, debtors often genuinely forget their obligations, forcing the creditor to send awkward manual follow-ups.

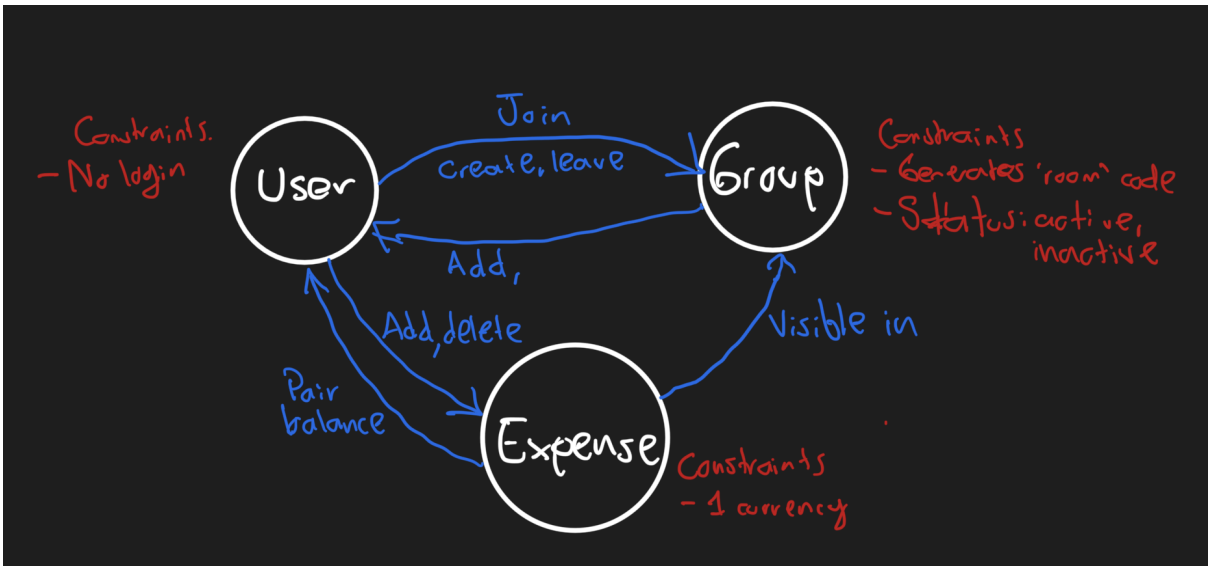
The Opportunity

There is a clear need for a **neutral, transparent, and automated intermediary** that removes the emotional weight from financial transactions, ensures mathematical precision, and provides a single "source of truth" for group finances.

Domain Model

Domain Model Description: Accountless Expense Splitter

The system is designed as a **decentralized, session-based expense manager** where the "Group" (or Room) acts as the single source of truth for all participants.



1. Core Entities

User (Participant)

In this MVP, a User is a transient entity representing a person within a group.

- **Constraints:** No authentication or global accounts. Identity is tied to the Group/Room session.
- **Responsibilities:** * Creates or joins a Group via a unique link/code.
    - Adds, modifies, or deletes Expenses.
    - Calculates "Pair Balances" (how much they owe or are owed by another specific user).
    - Can leave a group (assuming debts are settled).

Group (The "Room")

The Group is the central container for the shared experience.

- **Attributes:** Unique 'Room Code' (for entry), Status (Active/Inactive), and Member List.
- **Logic:** It manages the lifecycle of the shared session. It acts as the "database" where the expenses live.
- **Access:** Entry is granted via a shared URL or code, bypassing the need for a login database.

Expense

The record of a financial transaction.

- **Constraints:** Limited to **one currency** per group to simplify the MVP logic.
- **Attributes:** Amount, Description, Payer, and Split Method.
- **Logic:** Must be visible to everyone in the Group. It contains the logic for "Splitting" (how the total is distributed among participants).

---

2. Relationship Logic & Dynamics

- **Group Membership:** A User can "Join" or "Leave" a Group. When a User joins, they become a valid participant for any existing or future Expenses within that Room.
- **Expense Ownership:** An Expense is "Add/Deleted" by a User but belongs to the Group.
- **Balance Aggregation:** The system calculates a **Net Balance** for each User by iterating through all Expenses in the Group. This creates the "Pair Balance" view (A owes B, B owes C).

---

3. MVP Use Case Refinement

Entity	Action	MVP Business Rule
Group	**Create/Invite**	Generates a 6-digit code or unique URL.
Expense	**Split Charge**	At creation, the user selects who is involved and if the split is equal or custom.
Debt	**Calculate**	The system runs a simplification algorithm to minimize the number of transactions.
Debt	**Settle Up**	A virtual "payment" that balances the debt between two users without moving real money (yet).

---

4. Handling Edge Cases

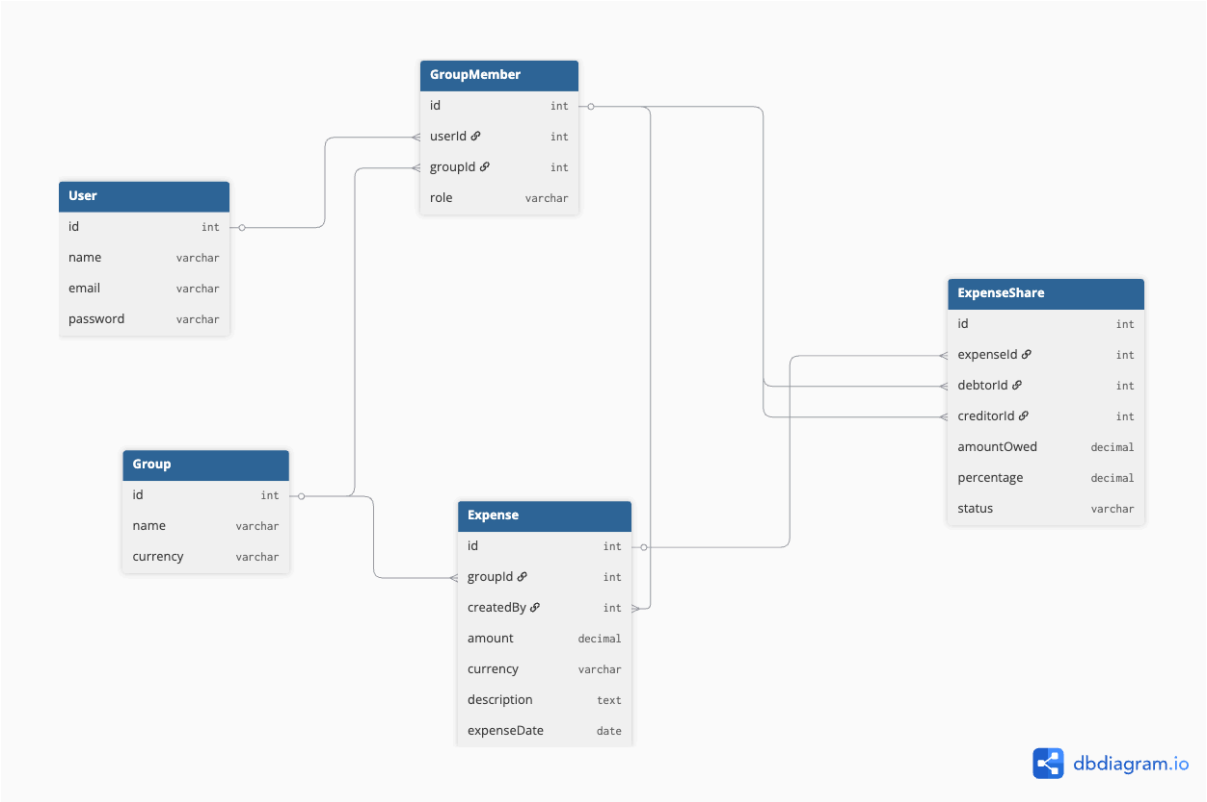
- **Late Joiners:** If a user joins after an expense is created, they aren't retroactively added to old debts unless the expense is manually edited.
- **The "Solo" Room:** If a user is alone in a group, they can record expenses, but the debt calculation remains at zero.
- **The "Runner" (Leaving with Debt):** The system should prevent a User from "Dropping Out" if their Net Balance is not zero, or at least trigger a warning to the Group.

Sprint 1: Implementation Focus

For your first sprint, the technical goal is the **CRUD (Create, Read, Update, Delete)** of the Room and the Expense:

1. **Room Logic:** Generate a unique ID and persistence (Local Storage or a simple NoSQL entry).
2. **User Identity:** Allow a user to set a "Display Name" when joining a room (stored in the browser session).
3. **Basic Split:** Implement 1-to-All equal splitting as the default behavior.

Domain Model Diagram



1. User

A real person using the application. A user can join groups, create expenses, pay or receive money, and communicate with other users. Every action in the system is performed by a user.

Attributes

- id
- name
- email
- password

Relationships

- Participates in many **Groups**

---

2. Group

A shared space where multiple users track expenses together, such as a trip, an apartment, or an event. All expenses, payments, and messages belong to a group.

Attributes

- id
- name

Relationships

- Contains many **Users**
- Contains many **Expenses**
- Contains many **Messages**

---

3. GroupMember (Association)

Represents the participation of a user in a group. It defines the user's role and permissions inside that group and forms the link between the user and group.

Attributes

- id
- userId (User)
- groupId (Group)
- role

Relationships

- Links one **User** to one **Group**
- Creates **Expenses**
- Sends and receives **Messages**
- Is responsible for **ExpenseShares**
- Sends and receives **Transactions**

---

4. Expense

A financial cost paid by one user and shared with one or more users in a group. Expenses represent actual past or current payments.

Attributes

- id
- groupId (Group)
- createdBy (GroupMember)
- type - is it for one member or split between multiple members
- amount

- currency
- description
- expenseDate
- createdAt

Relationships

- Belongs to one **Group**
- Created by one **GroupMember**
- Split into many **ExpenseShares**
- Can have many **Attachments**
- Can have many **Messages**

---

5. ExpenseShare

Represents the portion of an expense that a specific user is responsible for. It represents what one user owes to the other.

Attributes

- id
- expenseId (Expense)
- debtorId (GroupMember) - The person that must pay
- creditorId (GroupMember) - The person who must receive the money
- amountOwed
- percentage
- status - settled / pending

Relationships

- Belongs to one **Expense**
- Concerns two **GroupMember**

Use cases

Use cases: Golden scenarios

1. Group & Identity (The Onboarding)

- **The Creator's Journey:** User A arrives at the landing page, clicks "Create Group," enters the name "Weekend Trip," and is immediately redirected to a dashboard with a shareable URL.
- **The Seamless Join:** User B receives the URL on their phone, clicks it, enters the name "Charlie," and instantly sees the empty state of the group with User A already in the member list.
- **The "I'm Done Here" Exit:** A user with **zero balance** (owes nothing, owed nothing) clicks "Leave Group." They are removed from the member list, and the remaining members no longer see them as an option for future expenses.

2. Expense Creation & Splitting

- **The Standard "Equal Split":** User A pays $30 for a pizza. They select "Split Equally" between A, B, and C. The app should immediately show that B owes A $10 and C owes A $10.
- **The "I Paid for Two" (Custom Split):** User A pays $100 for groceries. User B and User C are a couple, so they agree to pay 2/3 of the bill. User A records the expense and manually assigns $66.66 to the couple and $33.34 to themselves.
- **The "Excluded Member" Scenario:** A group of 4 people has a "Beer Night." One person doesn't drink. When creating the expense, the payer unchecks the non-drinker. The app must divide the total only among the 3 participants who drank.

3. Debt Calculation & Settlement

- **The "Check My Math" Moment:** After 5 different expenses (some paid by A, some by B), User C goes to the "Balances" tab. The app should show a **single consolidated number**: "You owe User A $45."
- **The Virtual Handshake (Settle Up):** User B gives User A $20 in cash. User B clicks "Settle Up" in the app for that amount. User A receives a notification or sees the update, and both their balances update in real-time.
- **The "Total Group Cost" View:** The group admin looks at the dashboard and can see the **Total Spent** by the entire group to date, giving a sense of the "scale" of the trip or event.

4. Maintenance & Edits

- **The "Typo" Fix:** User A realizes they entered $50 instead of $500 for the hotel. They edit the expense. The app must **re-calculate the balances** for everyone involved in that specific expense automatically.

- **The Deletion Cleanup:** An expense was entered twice by mistake. User A deletes the duplicate. The balances must "roll back" to exactly what they were before the duplicate was added.

Use cases: Edge cases

1. Identity & Session Scenarios (The "No-Login" Challenge)

- **The "Incognito Guest" Trap:** User A creates a group and shares the link. User B opens the link in an **Incognito/Private tab**, adds an expense, and then closes the tab. When User B re-opens the link in a normal tab, can they still "claim" their identity, or did they just create a "Ghost User" that no one can settle up with?
- **The Duplicate Name Duel:** Two different people join the same room and both enter the name "Alex." When someone goes to split an expense, how does the app distinguish between "Alex (Payer)" and "Alex (Debtor)" to ensure the money goes to the right person?
- **The "Clear Cache" Disappearance:** User A has $50 in credit. They clear their browser cookies and site data. When they click the group link again, does the app recognize them, or are they forced to create a "new" user, leaving their $50 credit stranded on a "dead" profile?

2. The "Messy Dinner" (Expense Splitting)

- **The "Late to the Party" Adjustment:** A group of 4 people records a $100 dinner split equally ($25 each). A 5th person joins the room an hour later. Does the app offer a way to retroactively include them in that dinner split, or must the expense be deleted and recreated?
- **The "I Only Had a Salad" Split:** User A pays a $120 bill. Users B and C shared an expensive steak, while User D only had a side salad. Does the MVP allow User A to split the $120 **unevenly** (e.g., B and C owe $50, D owes $20), or does it only support equal division?
- **The Penny Conflict:** Three people split a $10.00 bill. Mathematically, it's $3.333... per person. Does your app charge one person $3.34 to balance the total, or does $0.01 simply vanish into the digital void?

3. Group Dynamics & Exit Strategies

- **The "Dine and Dash" (Leaving with Debt):** User B owes the group $40. User B clicks "Drop out from the group." Does the app allow this? If it does, does User B's

debt get distributed among the remaining members, or does the total balance of the group now fail to equal zero?

- **The "Solo Survivor":** Everyone leaves the room except for User A. User A tries to add an expense of $10. Since there is no one else to split with, does the app throw an error, or does it gracefully handle a "Debt to Self"?
- **The "Room Code" Brute Force:** If your room codes are 6-digit numbers (e.g., 123456), what happens if a stranger accidentally guesses the code and joins a private family vacation group? Is there a "Lock Room" feature?

4. Settlement & Calculation

- **The "Fake Settle" Dispute:** User B owes User A $20. User B clicks "Settle Up" in the app, but never actually sends the money via Venmo/Cash. Does User A have the power to **dispute or reject** that settlement, or is the debt wiped clean as soon as the debtor clicks the button?
- **The Debt Simplification Loop:**
    - A owes B $10.
    - B owes C $10.
    - C owes A $10. Does the app's "Calculate Debts" engine recognize that everyone is actually even and show $0 balance, or does it keep the circular debt alive?

MVP Sprints

Table of Content:

MVP Use Cases
- Group
- Expenses
- Edge Cases

Sprint 1: Completed

Sprint 2: In Progress
- Requirements Defined for Backend:

MVP Use Cases

Group

1. Create a group
2. Invite people to a group
3. Drop out from the group

Expenses

1. Create an expense
2. Split the charge of expense (during creation of the expense)
3. Settle up the debts
    a. **Calculate the debts!!!**
4. Drop out from an specific expense

Edge Cases

1. User added after creation of the group
2. Setting up the debt with a group of only one person in it
3. Person quitting the group without debt not settled up

Sprint 1: Completed

a. Define MVP Use Cases
b. Define Domain Model (Plain text)
c. Construct Domain.py file
d. Use cases:
    i. Create group
    ii. Invite people to a group
    iii. Create an expense with splitting
    iv. Settle up the debt
e. Create Test

Sprint 2: Completed

- [x] ~~Copy code and paste it in another repository (Fork the repository): Dias~~

Group 1: **Frontend Prototyping: Mikita & Slava**

- [x] ~~Create Mock/Prototype for UI~~
- [x] ~~Setup a FastAPI application, with API endpoints boilerplate, and generate documentation(swagger file, mock api server, authorization priority)~~
- [ ] After this, start with the frontend real application

Group 2: **API Logic Implementation: Hamza, Daniel**

- [x] ~~General structure of FastAPI Infrastructure~~
- [x] ~~Include domain files, services, repositories and tests in the FastAPI repo.~~
- [x] ~~Setup ORM (SQLAlchemy + Alembic, Using PostgreSQL)~~
- [x] ~~Create implementation logic of API endpoints~~

Group 3: **Simplifying the Settlement (User Requirement)**: **Dias, Sasha**

- [x] ~~Understands and Define the logic of the requirement~~
- [x] ~~Include in the Domain Models this requirement~~
- [x] ~~Implement tests for this requirement~~

Requirements Defined for Backend:

1. **Authentication & Users:** Handle user identity and session management.
   - [x] ~~POST /auth/register Register a new user account. { name, email, password, inviteGroupId }~~
   - [x] ~~POST /auth/login Log in and receive an access token (JWT). { email, password, inviteGroupId }~~
   - [x] ~~GET /auth/me Get the current authenticated user's profile. Headers: Authorization~~

2. **Group Management:** Manage the containers for expense sharing.
   - [x] ~~GET /groups List all groups the current user belongs to.~~
   - [x] ~~GET /groups/:code Get Group By Invite Code~~
   - [x] ~~POST /groups Create a new group. { name, type, currencyCode, coverImage }~~
   - [x] ~~GET /groups/:id Get metadata for a specific group.~~

3. **Group Members:** Manage who is in a group. Note: Adding a member implies they now have access to the ledger.
   - [x] ~~GET /groups/:id/members Get list of users in a group.~~
   - [x] ~~GET /groups/id/members/invite Get Invite Code~~
   - [x] ~~POST /groups/:id/members Add a user to a group (Join). { userId } or { email }~~
   - [x] ~~DELETE /groups/:id/members/:userId Remove a user from the group.~~

4. **Expenses:** The core transactional data. Note: Settlements are handled as expenses with a specific category.

- [x] ~~- GET /groups/:id/expenses List all expenses for a group (paginated). ?page=1&limit=50 DESC date~~
- [x] ~~- POST /groups/:id/expenses Create a new expense (or Settlement). { description, amount, payerId, splits: [...], category }~~
- [x] ~~- GET /expenses/:id Get details of a single expense.~~

5. **Debt management:** Manage the debt owed

- [x] ~~- GET /groups/debts List all debts in a group.~~
- [x] ~~- POST /groups/:id/debts/:id/settle Settle a debt.~~

Sprint 3: In Progress

Group 1: **Frontend Completion: Mikita**

- [x] ~~Finalization of Frontend Server, connected with Backend side~~
- [x] ~~Deployment~~

Group 2: **Backend Tests Completion: Hamza**

- [x] ~~Completion of Tests.~~
- [x] ~~Deployment~~

Final Project Checklist:

- [x] ~~**Non-technical criterias: Daniel and Dias**~~
  - [x] ~~User Requirements~~
  - [x] ~~Testing Plan~~
  - [x] ~~Technical Decision~~
  - [x] ~~Assumption and Constraints~~
  - [x] ~~Future Improvements~~
- [x] ~~**Technical Criterias: Hamza, Sasha, Slava**~~
  - [x] ~~Code Quality & Testing: Hamza~~
  - [x] ~~Pre-commit & Local Automation: Hamza~~
  - [x] ~~Continuous Integration (CI): Slava~~
  - [x] ~~Continuous Deployment (CD): Slava~~
  - [x] ~~Dependency management: Sasha~~
  - [x] ~~Repository hygiene: Sasha~~
  - [x] ~~Documentation: Sasha~~

User Requirements

User Requirements

1. User Account & Access

- As a user, I want to **create an account** (email / Google / Apple).
- As a user, I want to **log in and log out securely**.
- As a user, I want to **edit my profile** (name, photo, default currency).
- As a user, I want to **permanently delete** my account.

---

2. Groups & People

- As a user, I want to **create groups** (trip, apartment, event).
- As a user, I want to **add friends to a group** (email, username, link).
- As a user, I want to **add expenses with people**
- As a user, I want to **leave a group** when I'm done
- As an administrator (user that created the group), I want to **assign administrators to the group**.
- As a user, I want to write a message to the **group or specific person.**

---

3. Expenses

- As a user, I want to **add an expense** with:
  - Amount
  - Description
  - Date
  - Who paid
  - Currency
  - Who shares the expense (specific users of the group or the whole group)
- As a user, I want to **split expenses**:
  - Equally
  - By exact amounts
  - By percentages
- As a user, I want to **edit or delete an expense**.
- As a user, I want to **attach notes or photos** (receipts).
- As a user, I want to scan bills and get a list of products along with their prices to assign to buyers.
- As a user, I want to **see all the expenses and who added them** on a timeline
- As a user, I want to **set up recurring payments**
- As a user, I want to **make presets of recurrent payments**

---

4. Balances & Debts

- As a user, I want to **see my total balance** (who owes me / who I owe).
- As a user, I want to **see balances per group**.
- As a user, I want the app to **simplify debts automatically (Ex: Converting currency)**.

---

5. Notifications

- As a user, I want to be **notified when**:
  - Someone adds an expense
  - Someone edits or deletes an expense
  - Someone settles a payment
  - Reminders for payments that have not been settled yet.
  - I get invited to a new group
  - Someone leaves the group
  - My account is used from other IP
- As a user, I want to have reminders in my digital calendar.
- As a user, I want **to configure (turn on/off) email notifications.**

6. Payment Platform

- As a user, I want **to pay / settle a debt using payment platform (Paypal, VISA, MasterCard, others)**

7. Friends (Contacts)

- As a user, I want to **see a list of my contacts**.
- As a user, I want to **add new contacts**.
- As a user, I want to **create a reminder to my friend.**
- As a user, I want to **export a file with the expenses I share with a particular contact.**